

Análise Léxica e Sintática

Uma leitura integrada dos conceitos centrais da aula.

Visão geral

Este capítulo parte de análise do código fonte e avança até teste de disjunção par a par. No percurso, articula fnb na descrição da sintaxe, análise léxica, análise sintática descendente e lookahead. Em vez de tratar cada termo como uma definição isolada, a discussão mostra como os conceitos se conectam nas escolhas feitas por quem projeta, avalia ou utiliza linguagens de programação.

Análise do código fonte: critérios e conexões

O ponto de partida é **Análise do código fonte**: etapa utilizada nas implementações de linguagens de programação por compilação, interpretação pura ou sistemas híbridos. No escopo da aula, ela envolve a análise léxica e a análise sintática. A discussão avança para **FNB na descrição da sintaxe**: a FNB fornece uma descrição clara e concisa da sintaxe e pode servir de base para o analisador sintático, facilitando sua manutenção. Outro elemento dessa análise é **Analisador léxico e analisador sintático**: a análise de um processador de linguagens é dividida em analisador léxico, associado a autômato finito e gramática regular, e analisador sintático ou parser, associado a autômato com pilha e GLC ou FNB. Na prática, a análise passa por **Separação da análise léxica e da análise sintática**: a separação favorece a simplicidade, pois permite técnicas menos complexas no analisador léxico; a eficiência, por possibilitar sua otimização; e a portabilidade, por isolar partes lexicais dependentes da plataforma. Ao mesmo tempo, o quadro se completa com **Análise Léxica**: casamento de padrões que identifica subcadeias no código fonte, agrupa-as logicamente e fornece ao parser códigos internos correspondentes. O parser chama o analisador léxico quando precisa do próximo token.

Fontes: presentation, slide 3, paraphrase; presentation, slide 5, paraphrase; presentation, slide 4, paraphrase; presentation, slide 6, paraphrase; presentation, slide 7, paraphrase; presentation, slide 9, paraphrase

Análise Léxica: critérios e conexões

O ponto de partida é **Análise Léxica**: casamento de padrões que identifica subcadeias no código fonte, agrupa-as logicamente e fornece ao parser códigos internos correspondentes. O parser chama o analisador léxico quando precisa do próximo token. A discussão avança para **Lexema e token**: lexema é a subcadeia reconhecida no código fonte. Token é o código interno atribuído à categoria do lexema; por exemplo, diferentes nomes de variáveis podem receber IDENT, enquanto o símbolo de soma pode receber OP_SOMA. Outro elemento dessa análise é **Abordagens para construir o analisador léxico**: o analisador léxico pode ser obtido por uma ferramenta a partir da descrição formal dos tokens, implementado como programa baseado em um diagrama de estados ou construído por meio de uma tabela elaborada manualmente a partir desse diagrama. Na prática, a análise passa por **Classes de caracteres**: agrupamentos como letras e dígitos substituem transições individuais para cada caractere, reduzindo a extensão e a complexidade do diagrama de estados usado no reconhecimento de identificadores e inteiros. Ao mesmo tempo, o quadro se completa com **Reconhecimento de palavras reservadas**: palavras reservadas e identificadores podem ser reconhecidos inicialmente pelo mesmo padrão. Depois, uma tabela determina se o lexema reconhecido como possível identificador corresponde a uma palavra reservada. Por fim, esta parte se encerra com **Funções getChar, addChar e lookup**: getChar obtém o próximo caractere e determina sua classe; addChar incorpora o caractere ao lexema em processamento; lookup consulta a cadeia formada para verificar, entre outras possibilidades, se ela é uma palavra reservada e retornar seu código.

Fontes: presentation, slide 7, paraphrase; presentation, slide 9, paraphrase; presentation, slide 10, paraphrase; presentation, slide 11, paraphrase; presentation, slide 12, paraphrase; presentation, slide 15, paraphrase

FNB na descrição da sintaxe: critérios e conexões

O ponto de partida é **FNB na descrição da sintaxe**: a FNB fornece uma descrição clara e concisa da sintaxe e pode servir de base para o analisador sintático, facilitando sua manutenção. A discussão avança para **Objetivos da análise sintática**: o parser deve localizar erros sintáticos, apresentar diagnósticos e recuperar-se rapidamente, além de produzir a árvore de análise sintática ou ao menos um caminho correspondente nessa árvore. Outro elemento dessa análise é **Análise Sintática Descendente**: abordagem top down que constrói a árvore a partir da raiz, emprega derivação mais à esquerda e seleciona a regra do não-terminal corrente com base nos primeiros tokens que ele pode produzir. Na prática, a análise passa por **Análise Sintática Ascendente**: abordagem bottom up que começa pelas folhas e busca reduzir uma subcadeia correspondente ao lado direito de uma regra, reconstruindo no sentido inverso uma derivação mais à direita. A família LR é apresentada como a principal família de algoritmos ascendentes. Ao mesmo tempo, o quadro se completa com **Deslocamento e redução**: na análise sintática ascendente, deslocar move o próximo símbolo da entrada para a pilha; reduzir substitui uma subcadeia reconhecida pelo lado esquerdo da regra correspondente. Essas são as ações centrais dos analisadores shift-reduce da família LR. Por fim, esta parte se encerra com **Complexidade da análise sintática**: algoritmos aplicáveis a todas as gramáticas não ambíguas podem apresentar complexidade $O(n^3)$. Compiladores restringem-se a subconjuntos dessas gramáticas para realizar a análise em tempo linear $O(n)$, em que n é o tamanho da entrada.

Fontes: presentation, slide 5, paraphrase; presentation, slide 18, paraphrase; presentation, slide 19, paraphrase; presentation, slide 20, paraphrase; presentation, slide 22, paraphrase; sebesta-11ed, p. 202, paraphrase; sebesta-11ed, p. 204, paraphrase; presentation, slide 24, paraphrase

Análise Sintática Descendente: critérios e conexões

O ponto de partida é **Análise Sintática Descendente**: abordagem top down que constrói a árvore a partir da raiz, emprega derivação mais à esquerda e seleciona a regra do não-terminal corrente com base nos primeiros tokens que ele pode produzir. A discussão avança para **Análise Sintática Descendente Recursiva**: implementação formada por uma coleção de subprogramas, geralmente com um subprograma para cada não-terminal. Os subprogramas percorrem as regras e produzem a análise em ordem descendente. Outro elemento dessa análise é **FNBE para expressões simples**: a FNBE apresentada organiza expressões em *expr*, *term* e *factor*. *expr* trata soma e subtração entre termos; *term* trata multiplicação e divisão entre fatores; *factor* reconhece identificador, constante inteira ou expressão entre parênteses. Na prática, a análise passa por **Processamento do lado direito de uma regra**: para cada terminal no lado direito, o parser compara o símbolo com *nextToken* e sinaliza erro se não houver casamento. Para cada não-terminal, chama o subprograma correspondente. Quando um terminal casa, o analisador léxico fornece o próximo token. Ao mesmo tempo, o quadro se completa com **Convenção de nextToken**: cada rotina do analisador sintático deixa em *nextToken* o próximo token ainda não processado. Assim, a rotina seguinte pode examinar esse token para continuar a análise ou escolher uma regra. Por fim, esta parte se encerra com **Lookahead**: quando um não-terminal possui mais de uma regra, o próximo token da entrada é comparado com os primeiros tokens que cada alternativa pode gerar. Uma alternativa compatível é escolhida; a ausência de casamento caracteriza erro sintático.

Fontes: presentation, slide 19, paraphrase; presentation, slide 20, paraphrase; presentation, slide 26, paraphrase; sebesta-11ed, p. 208, paraphrase; presentation, slide 27, paraphrase; presentation, slide 28, paraphrase; sebesta-11ed, p. 192, paraphrase; presentation, slide 30, paraphrase; presentation, slide 31, paraphrase; sebesta-11ed, p. 189, paraphrase

Lookahead: critérios e conexões

O ponto de partida é **Lookahead**: quando um não-terminal possui mais de uma regra, o próximo token da entrada é comparado com os primeiros tokens que cada alternativa pode gerar. Uma alternativa compatível é escolhida; a ausência de casamento caracteriza erro sintático. A discussão avança para **Recursão à esquerda**: uma gramática com recursão à esquerda direta ou indireta não pode ser usada diretamente por um analisador sintático descendente. A transformação apresentada separa alternativas recursivas $A\alpha$ das alternativas não recursivas β e introduz um novo não-terminal A' . Outro elemento dessa análise é **FIRST**: $\text{FIRST}(\alpha)$ é o conjunto de símbolos terminais que podem aparecer no início de uma cadeia derivada de α . Se α puder derivar a cadeia vazia, ϵ também pertence a $\text{FIRST}(\alpha)$. Na prática, a análise passa por **Teste de Disjunção par a par**: para cada par de alternativas $A \rightarrow \alpha_i$ e $A \rightarrow \alpha_j$, os conjuntos $\text{FIRST}(\alpha_i)$ e $\text{FIRST}(\alpha_j)$ devem ter intersecção vazia. Se houver intersecção, o lookahead pode não determinar de forma única qual alternativa deve ser usada.

Fontes: presentation, slide 31, paraphrase; sebesta-11ed, p. 189, paraphrase; presentation, slide 36, paraphrase; presentation, slide 38, paraphrase; presentation, slide 39, paraphrase

Exemplos

Decomposição léxica de uma atribuição

Exemplo autoral: na entrada subtotal = quantidade * 25;, o analisador pode reconhecer os lexemas subtotal, =, quantidade, *, 25 e ; e associá-los a categorias como identificador, atribuição, multiplicação, constante inteira e delimitador.

Hierarquia sintática de uma expressão

Exemplo autoral: em preco + 8 * taxa, expr processa inicialmente um term; o segundo term contém a multiplicação entre dois fatores. Essa estrutura preserva a organização expr, term e factor apresentada na FNBE.

Detecção de parêntese ausente

Exemplo autoral: ao analisar (valor + 3, a rotina factor consome o parêntese esquerdo, chama expr e espera um parêntese direito. Se nextToken não corresponder ao fechamento, a rotina deve sinalizar erro sintático.

Pontos de atenção

Ao aplicar esses critérios, vale evitar alguns atalhos de raciocínio: Token e lexema não são sinônimos: o lexema é a cadeia encontrada, enquanto o token representa sua categoria interna. A análise léxica não substitui o parser; ela fornece ao parser os tokens reconhecidos. Separar análise léxica e sintática não é apenas uma decisão organizacional: a separação também favorece eficiência e portabilidade. A análise descendente começa pela raiz, enquanto a análise ascendente começa pelas folhas. Lookahead não significa examinar arbitrariamente todo o restante da entrada; no material, a decisão é apresentada com base no próximo token. Uma gramática com recursão à esquerda não pode ser usada diretamente por um analisador sintático descendente. Intersecção não vazia entre conjuntos FIRST de alternativas indica que o próximo token pode não ser suficiente para escolher uma regra. O trecho da rotina expr apresentado no material não realiza sozinho toda a detecção de erros sintáticos.

Bibliografia

Análise Léxica e Sintática. Material de aula em apresentação.

SEBESTA, Robert W. Conceitos de linguagens de programação. Tradução de João Eduardo Nóbrega Tortello. 11. ed. Porto Alegre: Bookman, 2018.

Este resumo privilegia paráfrases e articula os conceitos para leitura contínua. Consulte as fontes originais para contexto e aprofundamento.